Developing and Evaluating an Anomaly Detection System ⚙️

Building an effective machine learning system involves making many decisions (choosing features, setting parameters like ϵ). The process is significantly faster and more effective if you have a way to **evaluate** your system with a concrete number (a "real number evaluation").

Labeled vs. Unlabeled Data

Although anomaly detection is fundamentally an **unsupervised** learning technique (learning from normal data), evaluating it is much easier if we assume we have a **small amount of labeled data**.

- $y = 0$ **(Normal):** The vast majority of your data (e.g., 10,000 good engines).
- $y = 1$ **(Anomalous):** A small handful of known anomalies (e.g., 20 flawed engines you've seen in the past).

Splitting the Data

We split our data into three sets, ensuring that the **Training Set** consists only of "normal" examples (or examples we *assume* are normal), while the **Cross Validation (CV)** and **Test** sets contain a mix of normal examples and the few known anomalies.

Example Scenario:

- **Total Data:** 10,000 Good Engines ($y = 0$), 20 Anomalous Engines ($y = 1$).

Dataset Split	Composition	Purpose
Training Set	6,000 Good Engines ($y = 0$)	Used to fit the Gaussian model ($p(x)$). The algorithm learns what "normal" looks like.
CV Set	2,000 Good Engines ($y = 0$) 10 Anomalous Engines ($y = 1$)	Used to tune parameters (like ϵ) and select features.
Test Set	2,000 Good Engines ($y = 0$) 10 Anomalous Engines ($y = 1$)	Used for the final evaluation of the system's performance.

(Note: If you have very few anomalies—e.g., only 2—you might skip the Test Set and put all anomalies into the CV Set. This risks overfitting but is sometimes necessary.)

The Evaluation Process

1. **Train:** Fit the model $p(x)$ using the **Training Set**.
2. **Tune:** Use the **CV Set** to make predictions:
   - Calculate $p(x)$ for every example in the CV set.
   - Predict $y = 1$ if $p(x) < \epsilon$, and $y = 0$ otherwise.
   - **Adjust ϵ:** If the model misses the anomalies, increase ϵ. If it flags too many normal engines, decrease ϵ.
3. **Evaluate:** Once you are happy with the performance on the CV set, run the final model on the **Test Set** to get an unbiased estimate of its accuracy.

Metrics for Skewed Data

Because anomalies are rare (the data is highly "skewed"), standard classification accuracy is often a bad metric. (A model that always predicts "Normal" would have 99.9% accuracy but be useless).

Instead, use metrics like:

- **True Positive / False Positive / False Negative / True Negative** counts.
- **Precision and Recall**.
- **F1-Score**.

These metrics will tell you if your algorithm is actually finding the rare anomalies without falsely flagging too many normal examples.